



AES and SM4

Step-by-Step Encryption, Differential Fault Analysis, and Key Recovery

Educational Technical Notes for PFE 5A

Student: Guilherme Specht

Supervisor: Prof. Philippe Maurine

Institution: Polytech Montpellier

Industrial context: STMicroelectronics

Date: 7 November 2026

Contents

1 Purpose and Notation **1**

2 AES-128: Encryption from the Beginning **2**
2.1 Block, key, and round structure . . . 2
2.2 Initial AddRoundKey . . . 2
2.3 SubBytes . . . 2
2.4 ShiftRows . . . 3
2.5 MixColumns . . . 3
2.6 Round keys . . . 3

3 Piret–Quisquater Differential Fault Analysis of AES **4**
3.1 Where the fault is injected . . . 4
3.2 Propagation through MixColumns . . . 4
3.3 Which ciphertext bytes are affected? . . . 4
3.4 Constructing candidates for the last AES round key . . . 5
3.5 Intersecting observations . . . 5
3.6 Two injection depths: round 9 versus round 8 . . . 5
3.7 Reversing AES-128 key expansion . . . 6

4 SM4: Complete Structure and First Round **7**
4.1 Official test vector . . . 7
4.2 Encryption . . . 7
4.3 Nonlinear Transformation: S-box . . . 8
4.4 Linear Transformation . . . 8
4.5 Key Expansion . . . 8
4.6 Detailed computation of encryption round 0 . . . 10

5 Differential Fault Analysis of SM4: A Worked Last-Round Model **12**
5.1 Attack model and limitations . . . 12
5.2 Known ciphertext and hidden internal values . . . 12
5.3 Step 1: Inject a known XOR fault . . . 12
5.4 Step 2: Observe the S-box difference . . . 12
5.5 Step 3: Propagate the difference through L . . . 13
5.6 Step 4: Undo the linear layer from observed data . . . 13
5.7 Step 5: Convert S-box input candidates to key candidates . . . 13
5.8 Step 6: Use independent observations . . . 13
5.9 Step 7: Repeat for the other three bytes . . . 14
5.10 What happens after recovering the last subkey? . . . 14

6 SM4: Recovering the Master Key from Four Round Keys **15**
6.1 Required subkeys . . . 15
6.2 Reverse the recurrence . . . 15
6.3 Undo the initial constants . . . 15

7 Giraud Fault Attack on AES-128 and Its Adaptation to SM4 **16**
7.1 Core idea and comparison with Piret–Quisquater . . . 16
7.2 AES-128: precise fault location . . . 16
7.3 Worked AES-128 example: the first ciphertext byte . . . 16
7.4 Multiple faults and candidate intersection . . . 17
7.5 Two variants: known-bit and unknown-bit faults . . . 17
7.6 Adaptation of the single-bit idea to SM4 . . . 18
7.7 Worked SM4 example and recovery of rk_{31} . . . 18
7.8 From the final SM4 subkey to the master key . . . 19
7.9 Experimental implications . . . 20

8 Causing Single-Bit Faults in Practice **21**
8.1 Why a physical fault appears . . . 21
8.2 Common injection techniques . . . 21
8.3 Hitting the right instruction . . . 22
8.4 The gap between the model and reality . . . 22
8.5 A taxonomy of fault widths and their consequences . . . 22
8.6 Countermeasures . . . 24

9 AES versus SM4: Key Lessons **25**
9.1 The same attack on AES and on SM4? . . . 25
9.2 On SM4, what actually differs between Piret and Giraud? . . . 26

References and Scope **28**

Chapter 1

Purpose and Notation

This document reorganizes the supplied handwritten AES/SM4 notes into a progressive English explanation. It distinguishes established algorithmic facts, pedagogical fault simulations, and the additional work needed for a physical attack.

- $\oplus$: bitwise XOR; $\lll n$: 32-bit circular left rotation by n bits.
- S: an 8-bit substitution box (AES and SM4 use different S-boxes).
- C: correct ciphertext; F: faulty ciphertext; $\Delta = C \oplus F$.
- All hexadecimal words are written most-significant byte first. SM4 words are 32 bits.

Note. A known plaintext/key test vector is used to *verify* examples. In an actual key-recovery experiment, the attacker does not know the secret key.

Chapter 2

AES-128: Encryption from the Beginning

2.1 Block, key, and round structure

AES is a 128-bit block cipher. AES-128, AES-192, and AES-256 use 10, 12, and 14 rounds respectively. We focus on AES-128.

Plaintext	`00112233445566778899AABBCCDDEEFF`
Key K_0	`000102030405060708090A0B0C0D0E0F`
Ciphertext	`69C4E0D86A7B0430D8CDB78070B4C55A`

AES stores bytes in a 4×4 *column-major* state:

$$\begin{pmatrix} 00 & 44 & 88 & CC \\ 11 & 55 & 99 & DD \\ 22 & 66 & AA & EE \\ 33 & 77 & BB & FF \end{pmatrix}.$$

First apply AddRoundKey with K_0. Rounds 1–9 apply SubBytes, ShiftRows, MixColumns, AddRoundKey. Round 10 omits MixColumns.

2.2 Initial AddRoundKey

The state is XORed byte by byte with the first round key. For example, $\texttt{11} \oplus \texttt{01} = \texttt{10}$. The result is

$$\begin{pmatrix} 00 & 40 & 80 & C0 \\ 10 & 50 & 90 & D0 \\ 20 & 60 & A0 & E0 \\ 30 & 70 & B0 & F0 \end{pmatrix}.$$

2.3 SubBytes

Each byte is independently replaced through the AES S-box. Examples: $S(\texttt{00}) = \texttt{63}$, $S(\texttt{10}) = \texttt{CA}$, $S(\texttt{20}) = \texttt{B7}$, $S(\texttt{30}) = \texttt{04}$. The resulting state is

$$\begin{pmatrix} 63 & 09 & CD & BA \\ CA & 53 & 60 & 70 \\ B7 & D0 & E0 & E1 \\ 04 & 51 & E7 & 8C \end{pmatrix}.$$

The S-box is nonlinear: in general, $S(a \oplus b) \neq S(a) \oplus S(b)$.

2.4 ShiftRows

Rows 0, 1, 2, 3 rotate left by 0, 1, 2, 3 bytes respectively:

$$\begin{pmatrix} 63 & 09 & CD & BA \\ 53 & 60 & 70 & CA \\ E0 & E1 & B7 & D0 \\ 8C & 04 & 51 & E7 \end{pmatrix}.$$

2.5 MixColumns

Every column is multiplied over $GF(2^8)$ by

$$M = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}.$$

For the first input column $(\texttt{63}, \texttt{53}, \texttt{E0}, \texttt{8C})^T$,

$$b_0 = (02 \cdot 63) \oplus (03 \cdot 53) \oplus E0 \oplus 8C.$$

Multiplication is in the AES finite field, not ordinary integer arithmetic. For a separate illustrative column $(\texttt{DB}, \texttt{13}, \texttt{53}, \texttt{45})^T$, MixColumns yields $(\texttt{8E}, \texttt{4D}, \texttt{A1}, \texttt{BC})^T$. The first column of the official AES round-1 state becomes $(\texttt{5F}, \texttt{72}, \texttt{64}, \texttt{15})^T$.

2.6 Round keys

AES-128 expands the original 16-byte key into eleven 16-byte round keys $K_0, \ldots, K_{10}$. For this vector:

$$K_1 = \texttt{D6AA74FDD2AF72FADAA678F1D6AB76FE},$$

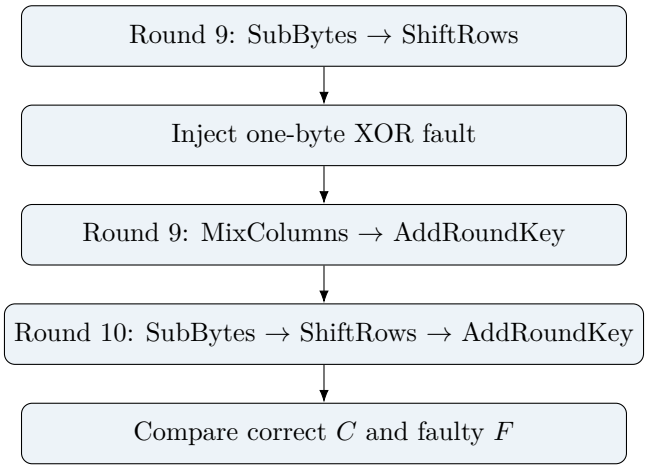
$$K_{10} = \texttt{13111D7FE3944A17F307A78B4D2B30C5}.$$

Chapter 3

Piret–Quisquater Differential Fault Analysis of AES

3.1 Where the fault is injected

In the classic model used in the notes, one nonzero byte is corrupted at the *input of MixColumns in round 9*, after SubBytes and ShiftRows. Both correct and faulty executions then proceed through round 9 and the final round.



3.2 Propagation through MixColumns

If the first byte of a column changes by e, the column difference after MixColumns is

$$\Delta_{MC} = (02e,\ e,\ e,\ 03e)^T.$$

For $e = \texttt{01}$, this is $(\texttt{02}, \texttt{01}, \texttt{01}, \texttt{03})^T$. AddRoundKey does not alter XOR differences because both executions use the same round key. The final SubBytes transforms these differences nonlinearly; ShiftRows moves them into separate ciphertext positions.

3.3 Which ciphertext bytes are affected?

For the four round-9 columns, the affected ciphertext positions (in corresponding row order) are:

Faulted column	Ciphertext byte positions
0	0, 13, 10, 7
1	4, 1, 14, 11
2	8, 5, 2, 15
3	12, 9, 6, 3

3.4 Constructing candidates for the last AES round key

For an affected ciphertext position j, and a guessed last-round key byte k, compute

$$d_j(k) = S^{-1}(C_j \oplus k) \oplus S^{-1}(F_j \oplus k).$$

Each byte has 256 candidate values. Four unknown key bytes have $256^4 = 2^{32}$ possible combinations before filtering. Construct a set $\mathcal{D}$ containing the MixColumns output differences for a single nonzero faulty byte in any of the four row positions:

$$\mathcal{D} = \{M(e\mathbf{e}_r) : r \in \{0, 1, 2, 3\},\ e \in \{1, \ldots, 255\}\}.$$

This produces $4 \cdot 255 = 1020$ four-byte difference patterns (not 1020 individual bytes). For each proposed key quartet, compute the four $d_j(k)$ values in their correct *pre-ShiftRows row order* and keep the quartet only if the resulting vector lies in $\mathcal{D}$.

3.5 Intersecting observations

A single correct/faulty pair typically leaves multiple candidate key quartets. Repeat with independent faults in the same column and intersect the candidate sets:

$$\mathcal{K}_{\text{surviving}} = \mathcal{K}_1 \cap \mathcal{K}_2 \cap \cdots .$$

Repeat across all four columns to recover the full 16-byte K_{10}. The notes illustrate candidate filtering using a hypothetical quartet (`13`, `2B`, `A7`, `17`); such pedagogical candidate examples should not be confused with a complete independently verified AES fault trace.

3.6 Two injection depths: round 9 versus round 8

The same Piret principle can be applied at two different depths, and the project's analysis distinguishes them because they behave very differently. The choice is the `fault_round` argument of the target in `piret_aes.py`.

Round-9 model (one column group). The fault is injected before the round-9 MixColumns, as described above. MixColumns spreads the single faulted byte over *one* column (four bytes); ShiftRows then scatters those four bytes to four ciphertext positions. One faulty ciphertext therefore constrains only *one* group of four last-round key bytes. To recover all sixteen bytes of K_{10}, the campaign must be repeated for each of the four columns.

Round-8 model (all four groups at once). The fault is injected one round earlier, before the round-8 MixColumns. Now the difference passes through *two* MixColumns layers: round-8 MixColumns spreads the single byte to a full column, and round-9 MixColumns spreads each of those bytes across all four columns. Before the final round the difference already touches all sixteen state bytes, so a single faulty ciphertext constrains *all four* key-byte groups simultaneously. This is why the measured campaigns recover the complete last-round key from a median of only two faulty ciphertexts.

The trade-off is depth versus count: the round-8 model needs far fewer faulty ciphertexts but assumes a cleaner, more deeply propagated fault and larger per-observation candidate structures; the round-9 model is simpler to reason about per column but must be repeated. The default command line and the analysis use the round-8 model; `run.py` follows the round-9 model used in the lecture notes. Both reduce to the same last-round key once enough observations are intersected.

3.7 Reversing AES-128 key expansion

Let $w_0, \ldots, w_{43}$ be 32-bit expanded-key words. For $i \geq 4$,

$$w_i = \begin{cases} w_{i-4} \oplus g(w_{i-1}), & i \equiv 0 \pmod 4, \\ w_{i-4} \oplus w_{i-1}, & \text{otherwise}, \end{cases}$$

where $g(w) = \text{SubWord}(\text{RotWord}(w)) \oplus \text{Rcon}$. In reverse, compute w_{i-4} by XORing w_i with the same known right-hand transformation. Begin with $w_{40}, \ldots, w_{43}$ from K_{10} and work backward to $w_0, \ldots, w_3$, yielding K_0. No inverse S-box is required for this reversal because the relevant inputs to g are already known.

Chapter 4

SM4: Complete Structure and First Round

4.1 Official test vector

Plaintext	`0123456789ABCDEFFEDCBA9876543210`
Master key	`0123456789ABCDEFFEDCBA9876543210`
Ciphertext	`681EDF34D206965E86B3E94F536E4246`

Split the plaintext into four 32-bit words:

$$X_0 = \texttt{01234567}, \quad X_1 = \texttt{89ABCDEF}, \quad X_2 = \texttt{FEDCBA98}, \quad X_3 = \texttt{76543210}.$$

4.2 Encryption

SM4 performs 32 rounds, producing one new 32-bit word per round:

$$\boxed{X_{i+4} = X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i)}.$$

Write $T = L \circ \tau$, combining byte substitution and linear diffusion. The output reverses the final four words:

$$C = X_{35} \parallel X_{34} \parallel X_{33} \parallel X_{32}.$$

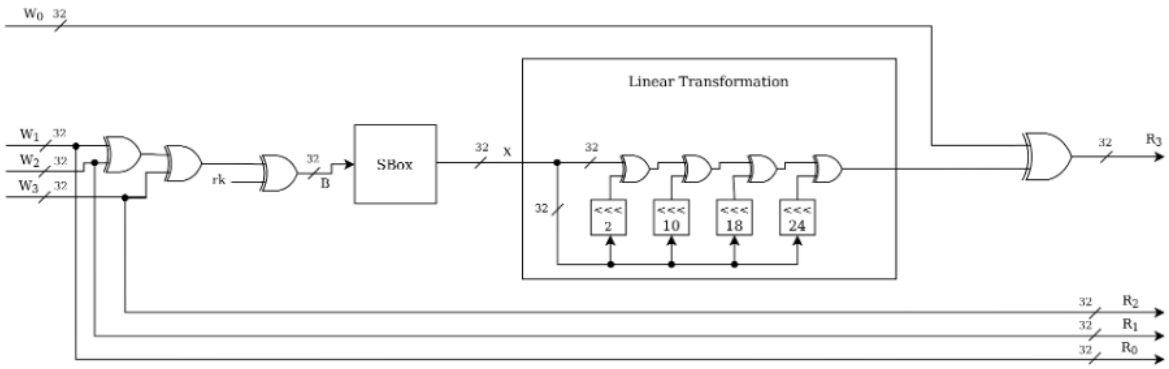


Figure 4.1: One SM4 encryption round. The four 32-bit input words advance by one position, and the newly computed word is appended.

The diagram uses $W_0, \ldots, W_3$ for the input window and $R_0, \ldots, R_3$ for the next window. Its intermediate X is the S-box output, denoted B below.

4.3 Nonlinear Transformation: S-box

The nonlinear transformation τ applies the same SM4 S-box independently to each of the four bytes of a 32-bit word:

$$\tau(A) = S(a_0) \parallel S(a_1) \parallel S(a_2) \parallel S(a_3), \quad A = a_0 \parallel a_1 \parallel a_2 \parallel a_3.$$

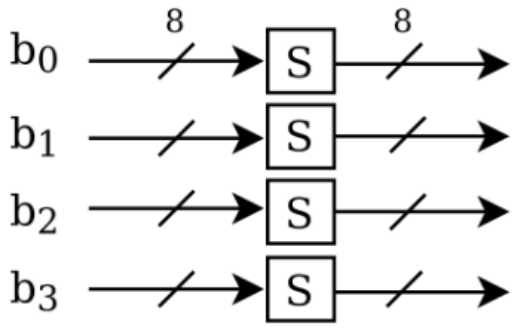


Figure 4.2: Four parallel 8-bit S-box substitutions form the nonlinear transformation.

4.4 Linear Transformation

After byte substitution, encryption combines the 32-bit word with four circular left rotations:

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24).$$

The arrows marked with rotation amounts represent circular rotations, not shifts that discard bits.

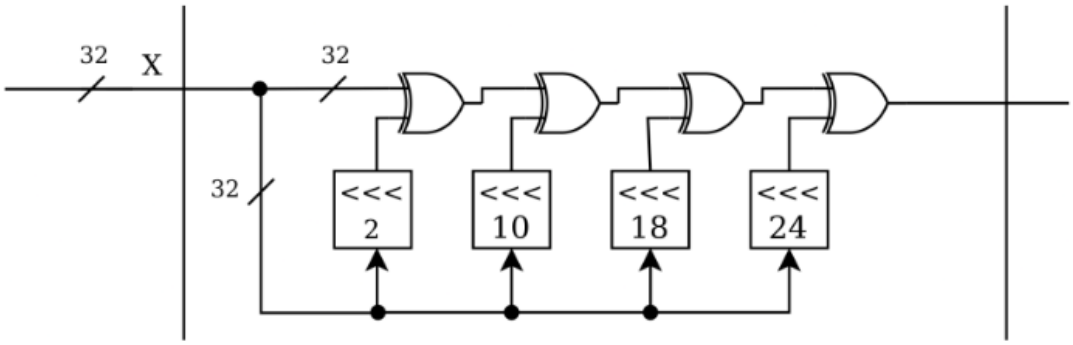


Figure 4.3: SM4 encryption diffusion: XOR of the original word and rotations by 2, 10, 18 and 24 bits.

4.5 Key Expansion

The four initial key-schedule words are $K_i = MK_i \oplus FK_i$, with

$$FK_0 = \texttt{A3B1BAC6}, \quad FK_1 = \texttt{56AA3350},$$
$$FK_2 = \texttt{677D9197}, \quad FK_3 = \texttt{B27022DC}.$$

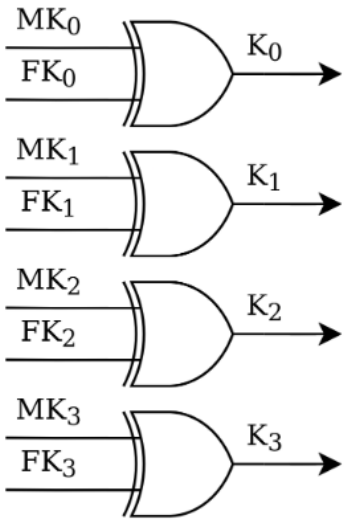


Figure 4.4: Initialization of the four key-schedule words by XOR with the fixed system parameters.

The key-schedule recurrence is

$$K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i), \quad rk_i = K_{i+4},$$

with $T' = L' \circ \tau$ and

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23).$$

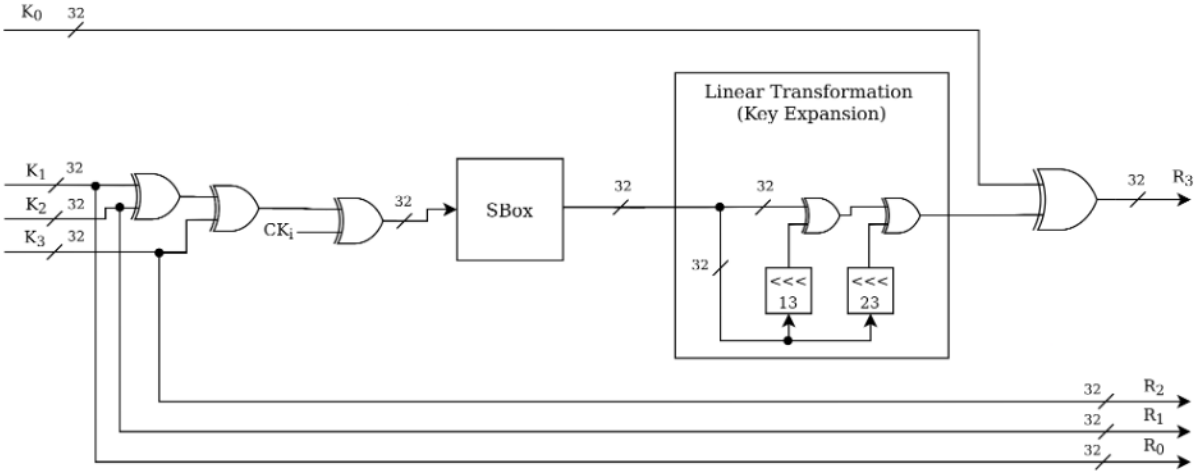


Figure 4.5: One SM4 key-expansion round. The round constant replaces the encryption round key, and the linear transformation uses rotations by 13 and 23 bits.

Crucial distinction: encryption uses L, while key expansion uses L'. They have different rotation constants. For the first round, $rk_0 =$ `F12186F9`.

4.5.1 How the public round constants are generated

SM4 defines 32 public 32-bit constants CK_i, one per key-expansion round. For $i = 0, \ldots, 31$ and byte index $j = 0, \ldots, 3$,

$$\boxed{ck_{i,j} = 7(4i + j) \bmod 256}, \qquad CK_i = ck_{i,0} \parallel ck_{i,1} \parallel ck_{i,2} \parallel ck_{i,3}.$$

Concatenation is big-endian. Reduction modulo 256 keeps each result within one byte; it is ordinary integer arithmetic, not multiplication in $GF(2^8)$.

Round i	Four bytes in decimal	CK_i in hexadecimal
0	0, 7, 14, 21	`00070E15`
1	28, 35, 42, 49	`1C232A31`
2	56, 63, 70, 77	`383F464D`
31	100, 107, 114, 121	`646B7279`

For example, $ck_{31,0} = 7 \times 124 \bmod 256 = 868 \bmod 256 = 100 =$ `64`. An implementation can compute these constants or store the same values in a table.

4.5.2 Where the system parameters FK come from

The four FK_i words listed above are fixed system parameters specified directly by SM4. The specification supplies their values rather than a CK-style generation formula. They initialize $K_i = MK_i \oplus FK_i$ for $i = 0, 1, 2, 3$; they are not 32 per-round constants.

Neither CK nor FK depends on the secret key or plaintext. The attacker knows both sets before collecting faults; recovering four consecutive final subkeys remains necessary for the schedule reversal described later.

Source: SM4 Internet-Draft, Sections 7.3.2–7.3.3 (public algorithm description; not an RFC).

4.6 Detailed computation of encryption round 0

Step 1: XOR the three newest words with the round key.

$$A = X_1 \oplus X_2 \oplus X_3 \oplus rk_0 = \texttt{F002C39E}.$$

Step 2: Apply the SM4 S-box to each byte.

$$\texttt{F0} \mapsto \texttt{18}, \quad \texttt{02} \mapsto \texttt{E9}, \quad \texttt{C3} \mapsto \texttt{92}, \quad \texttt{9E} \mapsto \texttt{B1}.$$

Thus $B = \tau(A) =$ `18E992B1`.

Step 3: Apply the linear transformation.

Term	Hexadecimal value
B	`18E992B1`
$B \lll 2$	`63A64AC4`
$B \lll 10$	`A64AC463`
$B \lll 18$	`4AC463A6`
$B \lll 24$	`B118E992`
XOR of all five	`26D99622`

Step 4: XOR with the oldest word.

$$X_4 = X_0 \oplus L(B) = \texttt{01234567} \oplus \texttt{26D99622} = \boxed{\texttt{27FAD345}}.$$

The next four-word window is (X_1, X_2, X_3, X_4). The following words are $X_5 = \texttt{A18B4CB2}$, $X_6 = \texttt{11C1E22A}$, and $X_7 = \texttt{CC13E2EE}$. After 32 rounds and the output reversal, the official ciphertext is obtained.

Chapter 5

Differential Fault Analysis of SM4: A Worked Last-Round Model

5.1 Attack model and limitations

We use a *controlled, known XOR fault* in one byte of the input to the SM4 S-boxes in round 31. This is a pedagogical model, not a claim that a real clock/voltage glitch yields a known XOR value. In this model, all other round inputs remain unchanged. The last round is

$$X_{35} = X_{31} \oplus L(\tau(A)), \qquad A = X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}.$$

The attacker observes the correct ciphertext and a faulty ciphertext. The secret internal values below are supplied *only for checking the simulation.*

5.2 Known ciphertext and hidden internal values

$$X_{35} = \texttt{681EDF34}, \quad X_{34} = \texttt{D206965E},$$
$$X_{33} = \texttt{86B3E94F}, \quad X_{32} = \texttt{536E4246}.$$

For this official test vector, the simulation has

$$X_{31} = \texttt{7B938F4C}, \quad A = \texttt{96FF9D45}, \quad rk_{31} = \texttt{9124A012}.$$

The attacker must recover the key rather than assume it.

5.3 Step 1: Inject a known XOR fault

Alter the first byte of A by $e = \texttt{01}$:

$$A = \texttt{96FF9D45} \longrightarrow A' = \texttt{97FF9D45}.$$

5.4 Step 2: Observe the S-box difference

The two S-box outputs are

$$B = \tau(A) = \texttt{1A488C73}, \qquad B' = \tau(A') = \texttt{55488C73}.$$

Consequently,

$$\Delta B = B \oplus B' = \texttt{4F000000}.$$

5.5 Step 3: Propagate the difference through L

Because L is XOR-linear,

$$\Delta X_{35} = L(B) \oplus L(B') = L(\Delta B) = \texttt{734E3D3D}.$$

The correct last word is $X_{35} = \texttt{681EDF34}$, so the faulty last word is

$$X'_{35} = \texttt{681EDF34} \oplus \texttt{734E3D3D} = \texttt{1B50E209}.$$

Therefore the attacker sees

Correct ciphertext	681EDF34	D206965E	86B3E94F	536E4246
Faulty ciphertext	1B50E209	D206965E	86B3E94F	536E4246
Difference	734E3D3D	00000000	00000000	00000000

5.6 Step 4: Undo the linear layer from observed data

The attacker computes

$$\Delta B = L^{-1}(\Delta X_{35}) = L^{-1}(\texttt{734E3D3D}) = \texttt{4F000000}.$$

The first S-box byte therefore satisfies

$$S(a) \oplus S(a \oplus \texttt{01}) = \texttt{4F}.$$

Enumerating all 256 possible input bytes gives the candidate set

$$\mathcal{A}_1 = \{\texttt{96}, \texttt{97}\}.$$

5.7 Step 5: Convert S-box input candidates to key candidates

The attacker knows the last three words from the ciphertext:

$$X_{32} \oplus X_{33} \oplus X_{34} = \texttt{07DB3D57}.$$

Since $rk_{31} = A \oplus \texttt{07DB3D57}$, the first byte of the key is

$$rk_{31}[0] = a \oplus \texttt{07}.$$

Thus $a = \texttt{96}$ yields $\texttt{91}$, while $a = \texttt{97}$ yields $\texttt{90}$. After one observation, the candidates are $\{\texttt{91}, \texttt{90}\}$.

5.8 Step 6: Use independent observations

Repeat with the *same plaintext and state*, but use a known XOR fault $e = \texttt{02}$. The new S-box difference is $d_2 = \texttt{81}$. Keep only input candidates satisfying both

$$\begin{cases} S(a) \oplus S(a \oplus \texttt{01}) = \texttt{4F}, \\ S(a) \oplus S(a \oplus \texttt{02}) = \texttt{81}. \end{cases}$$

The intersection identifies $a = \texttt{96}$, hence $rk_{31}[0] = \texttt{91}$.

5.9 Step 7: Repeat for the other three bytes

The supplied worked example reports the following bytewise observations under the same controlled-fault assumption:

Byte	d for $e = 01$	d for $e = 02$	Recovered A	Recovered rk_{31}
0	4F	81	96	91
1	71	83	FF	24
2	79	6F	9D	A0
3	80	C9	45	12

This reconstructs

$$A = \texttt{96FF9D45}, \qquad rk_{31} = A \oplus \texttt{07DB3D57} = \boxed{\texttt{9124A012}}.$$

Note. This illustrates last-round subkey recovery *conditional on the fault model*. The listed observations should be validated against a software implementation before treating them as experimental results.

5.10 What happens after recovering the last subkey?

Knowing rk_{31}, reconstruct the previous state word:

$$X_{31} = X_{35} \oplus T(X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}).$$

For the example, $X_{31} = \texttt{7B938F4C}$. However, this does *not* automatically reveal rk_{30}. Additional fault observations and state constraints are required for rounds 30, 29, and 28. The present worked example does not demonstrate that stage of the attack.

Chapter 6

SM4: Recovering the Master Key from Four Round Keys

6.1 Required subkeys

If four consecutive final subkeys have been recovered, then

Key-schedule word	Round key
K_{32}	$rk_{28} =$ `428D3654`
K_{33}	$rk_{29} =$ `62293496`
K_{34}	$rk_{30} =$ `01CF72E5`
K_{35}	$rk_{31} =$ `9124A012`

These are reference subkeys from the official example; *only* rk_{31} was recovered in the preceding pedagogical DFA walkthrough. Recovering the other three subkeys requires additional fault observations in earlier rounds and inversion of the already recovered later rounds.

6.2 Reverse the recurrence

The constants CK_i are public and generated by the byte formula given in the SM4 key-expansion chapter; they are not recovered by the attack. The forward key schedule is

$$K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i).$$

XOR is self-inverse, so

$$\boxed{K_i = K_{i+4} \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)}.$$

Start with $i = 31$, then decrease to $i = 0$. For example,

$$K_{31} = K_{35} \oplus T'(K_{32} \oplus K_{33} \oplus K_{34} \oplus CK_{31}),$$

where $CK_{31} =$ `646B7279`. This yields the previous expanded-key word $K_{31} =$ `F1780C81` in the official test vector. Continue to recover K_0, K_1, K_2, K_3.

6.3 Undo the initial constants

The FK_i values are the four public fixed system parameters, not unknown key material. Finally,

$$MK_i = K_i \oplus FK_i \qquad (i = 0, 1, 2, 3),$$

and concatenate the four master-key words:

$$\boxed{MK = \texttt{0123456789ABCDEFFEDCBA9876543210}}.$$

Chapter 7

Giraud Fault Attack on AES-128 and Its Adaptation to SM4

7.1 Core idea and comparison with Piret–Quisquater

Giraud's AES fault-analysis approach exploits a *single-bit* fault close to the last round. In contrast to the Piret–Quisquater example, which injects a byte fault before the ninth-round MixColumns and typically modifies four ciphertext bytes, a single-byte fault immediately before the final SubBytes affects only one ciphertext byte: the last AES round does not contain MixColumns. The attacker tests last-round key-byte candidates by checking whether the inverse-S-box input difference has Hamming weight one.

7.2 AES-128: precise fault location

Let U denote the state after round 9 AddRoundKey, just before round 10 SubBytes. A one-bit XOR fault modifies a single input byte a into $a' = a \oplus e$, where $e \in \{\texttt{01}, \texttt{02}, \texttt{04}, \texttt{08}, \texttt{10}, \texttt{20}, \texttt{40}, \texttt{80}\}$. Round 10 performs SubBytes, ShiftRows and AddRoundKey K_{10}; there is no MixColumns. ShiftRows relocates the changed byte without increasing the number of affected bytes.

For the affected ciphertext position j, with correct byte c, faulty byte f, and unknown last-round key byte k, the attacker evaluates

$$\delta(k) = S_{\mathrm{AES}}^{-1}(c \oplus k) \oplus S_{\mathrm{AES}}^{-1}(f \oplus k). \tag{7.1}$$

The candidate is compatible with the fault model if

$$\delta(k) \in \mathcal{E} = \{\texttt{01}, \texttt{02}, \texttt{04}, \texttt{08}, \texttt{10}, \texttt{20}, \texttt{40}, \texttt{80}\}. \tag{7.2}$$

Equivalently, the 8-bit value $\delta(k)$ must have Hamming weight exactly one. If the injected bit position is known, the condition can be made stronger: $\delta(k) = e$.

7.3 Worked AES-128 example: the first ciphertext byte

We use the official AES-128 test vector already introduced in this report:

$$P = \texttt{00112233445566778899AABBCCDDEEFF},$$
$$K_0 = \texttt{000102030405060708090A0B0C0D0E0F},$$
$$C = \texttt{69C4E0D86A7B0430D8CDB78070B4C55A}.$$

The first byte entering the final SubBytes is $a = \texttt{BD}$. Inject $e = \texttt{01}$ in that byte:

$$\texttt{BD} \oplus \texttt{01} = \texttt{BC}. \tag{7.3}$$

The AES S-box maps $S(\texttt{BD}) = \texttt{7A}$ and $S(\texttt{BC}) = \texttt{65}$. The first byte of K_{10} is $\texttt{13}$, so

$$c = \texttt{7A} \oplus \texttt{13} = \texttt{69},$$
$$f_1 = \texttt{65} \oplus \texttt{13} = \texttt{76}.$$

Thus the simulated faulty ciphertext is

$$F_1 = \texttt{76C4E0D86A7B0430D8CDB78070B4C55A}. \tag{7.4}$$

The attacker does not assume the value $\texttt{13}$: it is shown here solely to verify the example. For candidate $k = \texttt{13}$, equation (7.1) gives

$$S^{-1}(\texttt{69} \oplus \texttt{13}) = S^{-1}(\texttt{7A}) = \texttt{BD},$$
$$S^{-1}(\texttt{76} \oplus \texttt{13}) = S^{-1}(\texttt{65}) = \texttt{BC},$$
$$\delta(\texttt{13}) = \texttt{BD} \oplus \texttt{BC} = \boxed{\texttt{01}}.$$

The candidate is compatible with a single-bit fault, but one correct/faulty pair may leave several candidates.

7.4 Multiple faults and candidate intersection

Repeat the encryption with the same plaintext and key and introduce an independent single-bit fault at the same location. In the worked example, choosing $e_2 = \texttt{02}$ produces $a'_2 = \texttt{BF}$ and a second faulty first ciphertext byte $f_2 = \texttt{1B}$. For observation t, define

$$\mathcal{K}_t = \{k \in \{0, \ldots, 255\} : S^{-1}(c \oplus k) \oplus S^{-1}(f_t \oplus k) \in \mathcal{E}\}. \tag{7.5}$$

The surviving candidates are the intersection $\mathcal{K} = \bigcap_t \mathcal{K}_t$. For the illustrated pair of faults, the recovered first byte is $K_{10}[0] = \texttt{13}$. The number of observations required is not fixed: it depends on the fault model and the observed differences.

Repeating the experiment across all sixteen pre-final-SubBytes byte positions allows the attacker, under this model, to reconstruct all sixteen bytes of K_{10}. For the official vector,

$$K_{10} = \texttt{13111D7FE3944A17F307A78B4D2B30C5}. \tag{7.6}$$

The AES-128 key schedule can then be reversed to recover the original key K_0, as explained in the preceding AES key-expansion chapter.

7.5 Two variants: known-bit and unknown-bit faults

The project's analysis runs Giraud in two modes, and the difference is exactly how much the attacker knows about the injected fault. It changes the filter in equation (7.1), not the location of the fault.

Unknown-bit (`Giraud-unknown`). The attacker knows only that *some* single bit was flipped. A key candidate survives when $\delta(k)$ has Hamming weight one, i.e. $\delta(k) \in \mathcal{E}$, a set of *eight* allowed values. This is the weaker filter, so more faulty ciphertexts are needed before the intersection collapses to one byte.

Known-bit (`Giraud-known`). The exact mask e is known (for example because the simulation controls it, or a physical campaign has been characterised). The condition tightens to a *single* allowed value, $\delta(k) = e$. This is the strongest filter and converges fastest.

Both variants recover the full key; they differ in how many observations that takes. In the project's measured campaigns (ten random keys each, one byte per observation), the medians were:

Cipher	Variant	Verified recoveries	Faulty ciphertexts (median [min–max])
AES-128	Giraud-known	10/10	32 [32–32]
AES-128	Giraud-unknown	10/10	34.5 [32–37]
SM4	Giraud-known	10/10	32 [32–32]
SM4	Giraud-unknown	10/10	34.6 [32–38]

The known-bit variant settles at the structural minimum of thirty-two faults (two one-bit faults on each of the sixteen targeted bytes for AES, and the analogous count for SM4), while the unknown-bit variant occasionally needs a few extra faults because the eight-value filter leaves more candidates to eliminate. Contrast this with Piret in the round-8 model, which recovers the whole AES last-round key from a median of only two faulty ciphertexts (Section 3.6): Piret pays with a more diffused fault and heavier per-observation algebra, whereas Giraud uses the simplest possible fault (one bit, last round) but needs many of them.

7.6 Adaptation of the single-bit idea to SM4

The corresponding SM4 experiment is an *adaptation of the single-bit differential filtering principle*, rather than a claim that the original AES attack transfers unchanged. The SM4 last round is

$$X_{35} = X_{31} \oplus L(\tau(A)), \qquad A = X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}. \tag{7.7}$$

The output is $C = (X_{35}, X_{34}, X_{33}, X_{32})$. Inject a single-bit fault in one byte of A, just before the last-round S-boxes. Unlike AES, the SM4 linear map L diffuses the one-byte S-box output difference over the 32-bit word X_{35}. The other three ciphertext words remain unchanged *under the assumed isolated-fault model.*

Let F be the faulty ciphertext and $\Delta X_{35} = X_{35} \oplus X'_{35}$. Since L is linear and invertible, the attacker can compute

$$\Delta B = L^{-1}(\Delta X_{35}) = \tau(A) \oplus \tau(A'). \tag{7.8}$$

For an affected byte with S-box input a and unknown single-bit error e, candidate inputs satisfy

$$S_{\mathrm{SM4}}(a) \oplus S_{\mathrm{SM4}}(a \oplus e) = d, \quad e \in \mathcal{E}, \tag{7.9}$$

where d is the corresponding byte of ΔB. If e is known from the simulation, it is used directly; otherwise all eight one-bit masks must be tested. Multiple correct/faulty observations reduce the candidate set by intersection.

7.7 Worked SM4 example and recovery of rk_{31}

Using the official SM4 test vector,

$$P = \texttt{0123456789ABCDEFFEDCBA9876543210},$$
$$MK = \texttt{0123456789ABCDEFFEDCBA9876543210},$$
$$C = \texttt{681EDF34D206965E86B3E94F536E4246},$$

the simulated input of the last S-box layer is $A = \texttt{96FF9D45}$. A one-bit fault $e = \texttt{01000000}$ gives $A' = \texttt{97FF9D45}$. The substitution outputs are

$$\tau(A) = \texttt{1A488C73},$$
$$\tau(A') = \texttt{55488C73},$$
$$\Delta B = \texttt{4F000000}.$$

After the linear layer,

$$\Delta X_{35} = L(\texttt{4F000000}) = \texttt{734E3D3D}. \tag{7.10}$$

Therefore the faulty output is

$$F_1 = \texttt{1B50E209D206965E86B3E94F536E4246}. \tag{7.11}$$

From the observed difference alone, the attacker obtains $\Delta B = L^{-1}(\texttt{734E3D3D}) = \texttt{4F000000}$. With known fault mask `01` in the first byte, the candidate equation is

$$S_{\mathrm{SM4}}(a) \oplus S_{\mathrm{SM4}}(a \oplus \texttt{01}) = \texttt{4F}. \tag{7.12}$$

The first observation permits $a \in \{\texttt{96}, \texttt{97}\}$. A second simulated fault of mask `02` gives the first-byte S-box difference `81`. Intersecting both sets identifies $a = \texttt{96}$.

The three known ciphertext words yield

$$X_{32} \oplus X_{33} \oplus X_{34} = \texttt{07DB3D57}. \tag{7.13}$$

Thus the first last-round key byte is

$$rk_{31}[0] = \texttt{96} \oplus \texttt{07} = \boxed{\texttt{91}}. \tag{7.14}$$

The supplied worked example applies analogous observations to the remaining three S-box inputs, obtaining

$$A = \texttt{96FF9D45}, \qquad rk_{31} = A \oplus \texttt{07DB3D57} = \boxed{\texttt{9124A012}}. \tag{7.15}$$

The bytewise simulated differences are summarized below.

Byte	d for $e = 01$	d for $e = 02$	Recovered A	Recovered rk_{31}
0	4F	81	96	91
1	71	83	FF	24
2	79	6F	9D	A0
3	80	C9	45	12

Note. The numerical values in this subsection are simulated, not physical fault-injection measurements. In particular, knowledge of the exact XOR mask is a strong experimental assumption.

7.8 From the final SM4 subkey to the master key

Recovering rk_{31} is not equivalent to recovering the 128-bit master key. The SM4 key schedule needs four consecutive 32-bit round keys to be inverted. In the official test vector these are

$$rk_{28} = \texttt{428D3654}, \qquad rk_{29} = \texttt{62293496},$$
$$rk_{30} = \texttt{01CF72E5}, \qquad rk_{31} = \texttt{9124A012}.$$

The values rk_{28}, rk_{29} and rk_{30} are *reference values*, not keys shown to have been recovered by the two last-round faults. The two last-round faults do not recover these other keys. A multi-round extension requires additional faults in rounds 30, 29 and 28 and inversion of the already recovered later rounds.

With four consecutive round keys, set $K_{i+4} = rk_i$ and invert the SM4 recurrence:

$$K_i = K_{i+4} \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i). \tag{7.16}$$

Iterating backward yields K_0, K_1, K_2, K_3. Undo the fixed initial constants FK_i via $MK_i = K_i \oplus FK_i$ and concatenate the four words to recover the 128-bit master key.

7.9 Experimental implications

A software simulation can inject an exact XOR mask at a chosen intermediate value. On an STM32 target, a voltage or clock glitch may instead alter multiple bits, skip an instruction, affect control flow, or produce no useful fault. A practical evaluation should therefore log correct and faulty ciphertexts, classify observed fault patterns, verify whether the assumed location and single-bit model are plausible, and report recovery success separately from simulation-only results.

Chapter 8

Causing Single-Bit Faults in Practice

The attacks in this report assume a very specific event: a short, transient fault that changes one bit (Giraud) or one byte (Piret) of an intermediate value, at a chosen instruction, while leaving everything else correct. The mathematics then turns the correct/faulty ciphertext pair into key information. This chapter explains, at the level of a laboratory evaluation, how such faults are produced on real hardware, how far the clean single-bit model is from typical practice, and which countermeasures remove the attacker's advantage. It is written for a defensive security evaluation on the project's own ChipWhisperer target; it is not a procedure against any third-party device.

8.1 Why a physical fault appears

Digital logic is correct only when every signal settles within one clock period and every transistor is powered within its specified range. A fault-injection technique deliberately violates one of these guarantees for a very short time, so that a flip-flop latches a value that has not finished computing, or a memory cell flips. Because the violation is brief and localised, most of the computation stays correct and only a few bits are disturbed — which is exactly what differential fault analysis needs.

8.2 Common injection techniques

Clock glitching. When the attacker controls the device clock, inserting one abnormally short clock period forces a register to sample before the slowest combinational path has settled. The bits on the slow paths latch incorrectly. It is highly repeatable and is the method most able to approach a one- or few-bit fault, but it only applies to targets that run from an external, attacker-supplied clock. This is the mechanism exercised on the project's STM32 target.

Voltage glitching. A brief, sharp drop (or spike) on the supply rail slows gate switching so that, again, logic fails to settle in time. It needs no clock access and little equipment — a transistor pulling the rail low (a "crowbar") is enough — which makes it one of the most widely used techniques. Its spatial and bit-level precision is lower than clock glitching.

Electromagnetic (EM) pulse injection. A small coil held over the package emits a fast magnetic pulse that induces transient currents in nearby logic. It is contactless, needs no decapsulation, and can be aimed at a region of the die, giving some spatial selectivity.

Optical / laser injection. A focused light or laser pulse on an exposed (decapsulated) die deposits charge in specific cells. It is the most precise technique and can come closest

to a true, chosen single-bit fault, but it is invasive and requires expensive equipment and sample preparation.

Auxiliary stressors. Temperature and body-biasing shift timing margins and are sometimes combined with the above to widen the window in which a glitch succeeds.

8.3 Hitting the right instruction

Producing a disturbance is only half the problem; it must land on the targeted operation (for Giraud, the byte just before the final SubBytes). Two controls do this:

- A **trigger** marks when the sensitive computation runs. In this project the firmware raises a GPIO line (TIO4) around the cipher, and the capture hardware watches it. A real evaluation may instead trigger on a power or EM feature.
- A **delay** after the trigger selects *which* clock cycle, and therefore which instruction, receives the glitch; the glitch *width* and *offset* set its strength. An evaluation sweeps these parameters, repeats each setting many times, and classifies the outcome.

8.4 The gap between the model and reality

A real campaign does not produce clean single-bit faults on demand. Each attempt typically falls into one of several outcomes:

Observed outcome	Use for DFA
No effect (correct output)	none
Reset or crash	none
Instruction skip / control-flow change	different attack class
Multi-bit or multi-byte fault	usually breaks the single-fault model
Clean one-bit / one-byte transient fault	*this* is what Giraud/Piret need

The clean case is often a minority of attempts. The practical workflow is therefore statistical: log the correct ciphertext and every faulty ciphertext with its glitch parameters, *filter* for the faults whose difference matches the assumed model, and feed only those to the solver. This is why the software experiments in this report separate the target (which may expose internal state) from the solver (which sees only ciphertexts): the solver is written to accept exactly the observations a real, filtered campaign would yield.

Note. The numerical examples elsewhere in this report inject an exact XOR in software, so the fault model holds by construction. On the physical STM32 target the same model must be *established experimentally*: knowledge of the exact bit (the `Giraud-known` variant) is a strong assumption that a campaign has to justify by characterising its own glitches.

8.5 A taxonomy of fault widths and their consequences

A disturbed computation can land anywhere on a spectrum, from no effect to a corrupted whole word. What decides whether Giraud or Piret can use it is, above all, the *width* of the fault — how many bits or bytes actually changed. The two attacks are defined by the width they expect:

- **Giraud** expects *exactly one bit* flipped at the input of the last S-box layer.
- **Piret** expects *exactly one byte* corrupted (any nonzero value) before the round-9 MixColumns.

Consequence by fault width. This is the dimension that decides whether an attack *works* or *breaks*.

Fault width	Giraud (model: 1 bit)	Piret (model: 1 byte)
Zero bits (no effect)	$C = C'$, useless	$C = C'$, useless
One bit	*ideal*	works — one bit is a byte of Hamming weight one
2–8 bits in the **same** byte	breaks: input difference has weight > 1, the weight-one filter rejects the true key	*ideal* — this is exactly a single-byte fault
One whole byte	breaks (same reason)	*ideal*
≥ 2 different state bytes	breaks the single-fault model	breaks: pattern no longer matches one byte in a column

The central observation is that **Piret is a superset of Giraud**: every one-bit fault is also a one-byte fault (a byte value of Hamming weight one), so the Piret solver accepts a bit, whereas Giraud demands weight one and rejects anything wider. This is why, on SM4, the two attacks nearly coincide — the only difference is that Giraud assumes e is a single bit while Piret accepts the whole byte.

Consequence by fault position (unknown location). Not knowing *where* the fault landed does *not* break the attack; it is recovered by observation and raises the cost.

Situation	How it is resolved	Consequence
Unknown which column / key group	the set of changed ciphertext positions is the signature (see the column→positions table of the Piret chapter)	none — the group is *read* from $C \oplus C'$
Unknown exact position within the group	the solver sweeps all rows: $\mathcal{D} = \{M(e\,\mathbf{e}_r) : r \in \{0,1,2,3\},\ e \in \{1,\ldots,255\}\}$	weaker filter ⇒ more faults

For AES Giraud this is even more direct: a one-bit fault changes exactly one ciphertext byte (no MixColumns in the last round), so the changed position already names the attacked key byte.

Consequence by fault value (unknown pattern). Not knowing the exact value does not break the attack either; it only weakens the filter.

Attack	Value known	Value unknown	Measured
Giraud	knows which bit flipped: tests **1** value of e	only "some bit" (weight 1): tests **8** values (`01`,…,`80`)	$32 \to \approx 34.6$
Piret	already assumes any value — by construction	sweeps $e \in \{1,\ldots,255\}$	unchanged

Because Piret already treats the value as unknown, it has no known/unknown *value* variant — only the *position* can be known or not.

Consequence by timing (wrong round or instruction). If the glitch lands on the wrong round or instruction, the difference propagates through a different number of S-box layers and the pattern of changed bytes no longer matches the expected one:

- Too early ⇒ extra S-box layers scramble the difference ⇒ matches no pattern ⇒ filtered out.
- Too late (e.g. already at the output) ⇒ wrong positions change ⇒ filtered out.

The delay sweep of §8.5's sibling section exists precisely to place the glitch on the right cycle.

Summary.

What you do not know / what happened	Giraud	Piret	Action
No effect	useless	useless	discard
One bit, bit known	✓ 32 faults	✓ works	solve
One bit, bit unknown	✓ ≈34.6	✓ works	sweep e
Several bits, same byte	breaks	✓ ideal	Piret solver
One byte, pos. unknown	n/a	✓ more faults	read group, sweep r
Several bytes	breaks	breaks	discard
Wrong round / instr.	no match	no match	retune delay

The rule that ties it together: the less you know (value, position), the *weaker the filter* and the *more faults* you need — but the attack stays valid. What actually *breaks* an attack is the width leaving the model: more than one byte breaks both, while more than one bit breaks only Giraud (Piret still holds).

8.6 Countermeasures

Because the attacks depend on obtaining a clean faulty ciphertext that matches a known model, the defences aim to deny exactly that:

Redundancy and verification. Compute the cipher twice, or decrypt the result and compare; release the output only if both agree. A single transient fault is then detected and suppressed.

Infection. If a difference is detected, scramble the output so the faulty ciphertext carries no usable single-byte relationship instead of the clean difference the solver needs.

Sensors and detectors. Clock, voltage, EM and light sensors reset the device when an injection is attempted.

Randomisation. Masking and random timing make it harder to aim a fault at a specific intermediate value and to interpret the resulting difference.

None of these is implemented in this project's deliberately unprotected educational target; their absence is what makes the target suitable for measuring the attacks in the first place.

Chapter 9

AES versus SM4: Key Lessons

Property	AES-128	SM4
Block size	128 bits	128 bits
Key size	128 bits	128 bits
Rounds	10	32
State	4×4 byte matrix	Four 32-bit words
Nonlinearity	AES S-box	SM4 S-box
Linear diffusion	MixColumns and ShiftRows	Rotations and XOR in L
Last-round target	K_{10}, 128 bits	rk_{31}, 32 bits
Master-key recovery	Reverse AES expansion	Reverse SM4 expansion from four consecutive round keys

9.1 The same attack on AES and on SM4?

The *principle* of each attack carries over to SM4 — inject a fault before a non-linear S-box layer, read it out through the (invertible) linear layer, and filter key candidates with an S-box differential — but the *structure* differs enough that SM4 is not simply a copy of the AES case.

Piret: AES versus SM4.

	AES-128 Piret	SM4 Piret
Fault	one byte before a MixColumns	one byte at an S-box input
Linear layer exploited	MixColumns (mixes 4 bytes of a column)	L (mixes the 4 bytes of a 32-bit word)
How it is solved	match the MixColumns differential over a 4-byte key group	invert L, then solve the S-box differential byte by byte
Faults needed (median)	≈ 2 (round-8 model)	≈ 32
Recovered per campaign	all of K_{10} (128 bits)	one subkey rk_i (32 bits)

Giraud: AES versus SM4.

	AES-128 Giraud	SM4 Giraud
Fault	one bit before final SubBytes	one bit at an S-box input
Last-round linear layer	**none** (AES last round has no MixColumns)	L is still present
Output effect	exactly one ciphertext byte changes	a full 32-bit word changes
How it is solved	filter directly by Hamming weight one	invert L first, then filter by Hamming weight one
Faults needed (median)	32 (known) / $\approx$ 34.5 (unknown)	32 (known) / $\approx$ 34.6 (unknown)

The key point. On *AES*, Piret and Giraud are genuinely different: different round, byte fault versus bit fault, and two faulty ciphertexts versus thirty-two. On *SM4* they move much closer together, which is probably why they can look like "the same thing". The reason is structural: SM4 applies its linear layer L in *every* round, including the last, so both SM4 attacks must first invert L and then solve the same S-box differential. What is left to distinguish them is only the fault they assume — a known whole-byte XOR for the Piret model, or a single bit (known or unknown) for Giraud. In other words, on SM4 the two attacks share one skeleton and differ only in the fault model, while on AES the absence of MixColumns in the last round gives Giraud a shortcut that Piret does not have.

9.2 On SM4, what actually differs between Piret and Giraud?

Because SM4 keeps its linear layer L in every round, both attacks run the *same five steps*. Given a correct ciphertext and a faulty one for the same input:

1. Inject the fault at an **S-box input** of the last round.
2. Invert the linear layer: $\Delta B = L^{-1}(\Delta X_{35}) = \tau(A) \oplus \tau(A')$.
3. Solve the S-box differential byte by byte: keep the inputs a with $S_{\mathrm{SM4}}(a) \oplus S_{\mathrm{SM4}}(a \oplus e) = d$.
4. Convert the surviving a into bytes of the round key rk_{31}.
5. Repeat for rounds 30, 29 and 28, then invert the key schedule.

The implementation makes this literal: `giraud_sm4.py` reuses the target and the L^{-1} routine of `piret_sm4.py`. The *only* thing that changes is the fault assumed in step 1, which in turn sets the value of e used in step 3:

	SM4 Piret	SM4 Giraud
Fault injected	one **whole byte**, value known	one **single bit**
Mask e in $S(a) \oplus S(a \oplus e) = d$	a known byte, `01`–`FF`	a one-bit mask (`01`,`02`,...,`80`), known or not
Everything else (steps 2–5)	identical	identical

The measured faulty-ciphertext counts confirm it: SM4 Piret and SM4 Giraud-known both need a median of **32** faults — the same procedure, the same cost — driven by SM4's structure (four bytes per subkey, four subkeys to peel). Only Giraud-*unknown* rises slightly (median $\approx$ 34.6)

because not knowing which bit flipped leaves eight possible values of e instead of one, a weaker filter in step 3. So on SM4 the honest statement is: *Piret and Giraud are the same attack, differing only in whether the injected fault is a known whole byte or a single bit.* On AES that is not true, because AES has no MixColumns in the last round and Giraud exploits exactly that.

Cost of the master key. A further difference is how much a successful campaign buys. Reversing the AES key schedule needs the single 128-bit last-round key K_{10}, so one campaign plus a schedule inversion recovers the master key. The SM4 schedule instead needs *four consecutive* 32-bit round keys $(rk_{31}, rk_{30}, rk_{29}, rk_{28})$, so the attacker must mount four successive campaigns — faulting rounds 31, 30, 29 and 28 in turn and peeling off each recovered subkey — before the master key can be reconstructed. SM4's extra rounds make each attack recover proportionally less per fault.

Summary

For AES Giraud, inject a one-bit fault before final-round SubBytes, test inverse-S-box differences for Hamming weight one, and intersect key-byte candidates. The corresponding SM4 adaptation first inverts the linear map L to isolate the S-box differential.

For AES Piret–Quisquater, compare correct and faulty ciphertexts, invert the last S-box under candidate K_{10} bytes, match round-9 MixColumns differential patterns, intersect candidates, and reverse the AES key schedule.

For the illustrated SM4 model, compare correct and faulty ciphertexts, invert the linear map L, test S-box input candidates using the known fault differences, derive rk_{31}, and obtain further subkeys with additional appropriate experiments. Once four consecutive round keys are known, the SM4 key schedule can be reversed exactly.

References and Scope

1. Student-provided handwritten notes, *AES/SM4 notes*, 27 pages, including AES fundamentals, Piret–Quisquater, SM4 fundamentals, and an SM4 DFA example.
2. NIST, *FIPS 197: Advanced Encryption Standard (AES)* (algorithm and official AES test vector).
3. ISO/IEC 18033-3, *Encryption algorithms — Part 3: Block ciphers* (SM4 specification); SM4 test vector as reproduced in the supplied notes.

Note. The SM4 DFA chapter uses a deliberately strong fault assumption: known XOR magnitude at a specified S-box input. Experimental fault injection must establish whether that assumption is justified. Full recovery requires additional observations from earlier rounds; no full physical recovery of all four SM4 final round keys is claimed here.